

COMP4347 Individual Contribution Reflection

Anthony Doan

1 Subsystem Responsibility and Implementation

I was responsible for the **quiz-taking and review subsystem** of the MERN quiz application. This covered the quiz database models, backend routes/controllers, marking logic, JWT authentication integration, and frontend pages for taking quizzes, viewing history, and reviewing attempts.

I built the **Question** and **Score** models. **Question** stores question text, four options, the correct answer index, an optional explanation, and an **isActive** flag. **Score** stores each completed attempt, including the authenticated user ID, selected answers, correctness indicators, score, and timestamps. Together, these models support the approved **review mode after completion** variation.

I implemented **GET /api/quiz/questions**, **POST /api/quiz/submit**, **GET /api/quiz/history**, and **GET /api/quiz/review/:scoreId**. The question endpoint hides **correctAnswer** so users cannot inspect the answer key before submission. The submit endpoint receives selected answer indices, fetches correct answers from MongoDB, marks the attempt, calculates the score, and saves the result.

On the frontend, I implemented the quiz API helper, protected route, quiz page, history page, and review page. Users can load active questions, answer them, submit the quiz, view their score, and review past attempts. I also added animated answer cards, progress/status panels, a result card, a locked completed-attempt state to prevent repeat submissions, and a combined animated login/register landing page using the existing authentication API.

2 Major Technical Challenge and Resolution

The main technical challenge was integrating my quiz subsystem with my teammate's JWT authentication system. Initially, I tested quiz submission with a temporary fake **userId**, but this had to be replaced once attempts needed to belong to logged-in users.

My controller expected the user ID in fields such as **req.user.id** or **req.user._id**, but the decoded JWT stored it as **req.user.userId**. This caused protected quiz submission to fail with **User ID is required**, even though the frontend sent a valid token.

I fixed this by adding a helper function to safely extract the authenticated user ID:

```
const getUserIdFromRequest = (req) => {
  return (
    req.user?.id || req.user?._id || req.user?.userId ||
    req.user?.user?.id || req.user?.user?._id ||
    req.user?.data?.id || req.user?.data?._id
  );
};
```

I then updated **submitQuiz**, **getQuizHistory**, and **getQuizReview** to use this helper, and protected the submit, history, and review routes with authentication middleware. This tied attempts and review data to the correct authenticated user instead of a manual ID.

3 Mermaid Sequence Diagram

Figure 1 shows the main quiz-taking and review workflow: login, fetching active questions, submitting answers, backend marking, score saving, and review.

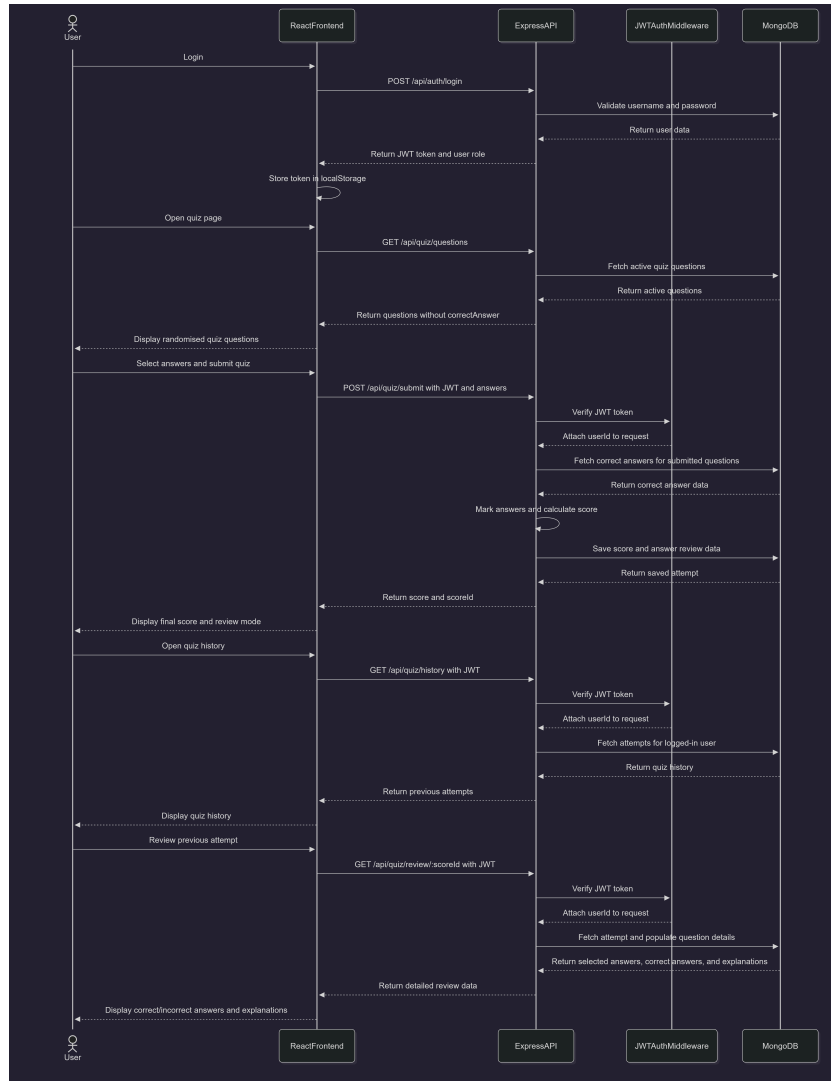


Figure 1: Mermaid sequence diagram for the user quiz-taking and review subsystem.

4 Git Commit History Analysis

I made consistent commits focused on the quiz-taking and review subsystem. The table below summarises the main evidence of my individual contribution.

Commit Hash	Contribution
b031ce3	Added core Mongoose models for quiz questions, scores, attempts, answers, and review data.
6e484b1	Initialised backend npm setup and gitignore configuration for backend development.
33eabd1	Created the initial quiz API route and controller structure.
905c134	Implemented secure question fetching by excluding <code>correctAnswer</code> from frontend responses.
89aaa00	Added quiz submission, backend marking logic, score calculation, and attempt saving.
48b3fe0	Added backend support for quiz history and review flow.
c9b366f	Integrated quiz API with JWT authentication so attempts are linked to logged-in users.
b0c168f	Built the frontend quiz-taking page with question loading, answer selection, submission, and score display.
aa4bec0	Improved navigation and protected the quiz route from unauthenticated users.
1d243c7	Added quiz history and review pages showing previous attempts and correct/incorrect answer details.
a501fed	Randomised all active quiz questions based on the admin-created active question set.
ef7945b	Polished the quiz UI with animated answer cards, side panels, progress styling, and locked completed attempts to prevent duplicate submissions.
9e5dcf2	Added a combined animated authentication landing page for login and registration while continuing to use the existing auth API.
6ffb0b7	Cleared authentication form fields after registration to avoid auto-filling login details and improve privacy on shared devices.

5 Reflection on Design Decisions and Group Variation

A key design decision was keeping quiz marking on the backend. The frontend only sends selected answer indices, while the backend retrieves correct answers from MongoDB. This prevents users from inspecting frontend responses to obtain the answer key.

I also chose to store completed attempts in the `Score` model instead of only displaying a temporary result. This enables persistent history, detailed review mode, and future features such as a leaderboard. I also locked the quiz after submission to prevent duplicate submissions from the same completed attempt.

Overall, I implemented the approved **review mode after completion** variation. Logged-in users can take a quiz, submit answers, receive a score, and review completed questions with selected answers, correct answers, correctness indicators, and explanations. The final UI separates quiz-taking, progress tracking, result display, and review actions clearly.